Improving AlemDICE: A Research Review and Experimental Roadmap

Repository technical report

21 July 2026

Abstract

Alem shows that frontier language models can make base-task progress while still failing at coordination, especially when actions must be synchronized. This report reviews the published Alem study, related work in LLM-agent coordination, cooperative AI, long-horizon agent design, evaluation methodology, and LLM serving. It then audits the current AlemDICE harness and proposes a prioritized improvement program. The highest-value near-term change is not a larger prompt or a central planner. It is an explicit, decentralized commitment protocol backed by structured plan memory, event-triggered messages, deterministic action checks, and paired-seed evaluation. The report also recommends partner-generalization tests, crash-stop experiments, and a serving study that separates policy quality from infrastructure failures. Every proposal preserves an unchanged published baseline and is framed as a falsifiable ablation rather than an assumed improvement.

1 Scope and method

The object of improvement is the complete text-agent system: observation and action interface, reasoning policy, communication, memory, execution scheduling, evaluation, and local inference. The environment's scientific value depends on keeping the canonical three-agent Alem control runnable. Consequently, recommendations in this report are new experiment profiles; none should silently change the leaderboard protocol.

The review uses four evidence classes. First, it reads the complete 42-page Alem preprint and its appendices [9]. Second, it examines the current repository configuration, evaluator, agent parser, communication tracker, team-leader treatment, provenance ledger, and Ollama provider. Third, it compares findings with primary work on coordination benchmarks, partner generalization, long-horizon agents, and statistical evaluation [2, 12, 8, 5, 1]. Fourth, it considers serving evidence and the observed four-A100 deployment [4, 7, 6]. Claims explicitly distinguish published results, repository facts, and proposed hypotheses.

2 What the evidence says

2.1 The benchmark exposes a real coordination gap

Alem combines procedural worlds with coordination requirements spanning delayed dependencies, handovers, and same-timestep joint action. Thirteen homogeneous LLM teams average only about 6% normalized return. The strongest base-task model is not necessarily the strongest coordinator: GPT-5.4-High has strong base reward but substantially weaker coordination reward, while Gemini-3.1-Pro-High approaches a one-billion-step MARL reference on Hard coordination [9]. This supports reporting base and coordination outcomes separately.

The most diagnostic failures are synchronous-hard tasks. Success requires discovering a common target, navigating to compatible positions, communicating roles and timing, and issuing compatible actions on the same tick. Even the best evaluated LLM covers only a minority of such rewards. Handover tasks are easier because their time window tolerates imperfect agreement. A productive harness should therefore target agreement and execution timing, not merely improve navigation.

2.2 Communication helps, but free-form broadcast is inefficient

Removing communication produces the largest coordination drop in the paper: Gemini coordination falls from 17.5% to 5.3%, and Gemma from 8.8% to 3.8% on Hard [9]. Yet the qualitative analysis shows that messages are mostly intent announcements and commands; questions and acknowledgements are rare. Gemma messages often behave as local status reports. Free-form broadcast therefore transmits useful information without guaranteeing that recipients formed the same commitment.

This is consistent with LLM-Coordination, which finds weaknesses when success depends on modeling partners' beliefs and joint plans rather than environmental facts [2]. MultiAgentBench finds that topology and cognitive planning affect milestone completion [12]; Collab-Overcooked similarly argues for process metrics because goal interpretation can look strong while active collaboration and adaptation remain weak [8]. Together these results favor measuring delivered coordination state, not raw message volume.

2.3 Memory helps only when it preserves plans

The Alem scratchpad ablation is model-dependent. Gemini's entries frequently preserve future-oriented, multi-line, sometimes turn-indexed plans, and removing memory hurts its coordination. Gemma's entries are usually one-line coordinate-heavy state summaries; removing them has little measured effect. Disabling Gemma reasoning makes its scratchpad longer and more deliberative, but does not recover performance [9]. More tokens are therefore not the right objective. Memory should preserve commitments, unresolved requests, discovered facts, and failure evidence while avoiding facts already present in the observation.

ReAct supports tight interleaving of reasoning and grounded action [11], while Voyager demonstrates the utility of persistent, executable skill abstractions in an open-ended world [10]. For AlemDICE, the conservative lesson is to build a small typed plan ledger and a tested skill layer before attempting unconstrained cross-episode reflection.

2.4 Heterogeneity and centralization are not free wins

The paper's mixed-model teams perform near the average of their constituent homogeneous teams, not the strongest member [9]. Different planning horizons and message conventions can create new coordination costs. A bodyless leader can make assignments, but it also introduces a privileged aggregation point, serial latency, extra tokens, and a single point of failure. The current AlemDICE leader treatment is scientifically useful as a comparison; it should not replace the decentralized headline.

Partner diversity remains important. Overcooked studies zero-shot coordination with new partners [3], and Melting Pot makes generalization to new co-player policies a first- class evaluation goal [5]. A robust AlemDICE policy must work with unfamiliar message styles and competence levels, not only three copies of itself.

3 Audit of the current AlemDICE baseline

Area	Current strength	Limitation or confound
Environment	Procedural coordination, soft specialization, three difficulty levels, matched seeds.	Most LLM runs die before advanced tiers; aggregate score can hide opportunity exposure.
Observation	Compact text, affordances, progressive disclosure, local history.	Spatial state remains prose-heavy; no explicit uncertainty or change set; possible leakage needs systematic audit.
Action	Canonical action list, tagged parser, fallback to Noop, parse metrics.	Syntactic validity is checked after generation; semantic feasibility and repeated-failure recovery are weak.
Communication	Parallel worker calls, delayed peer broadcast, byte and lexical metrics.	Broadcast is unstructured; no recipient, message type, commitment ID, expiry, acknowledgement, or conflict resolution.
Memory	Private bounded scratchpad and reasoning history.	Free text encourages state restatement; no durable goal, commitment, evidence, or confidence fields.
Planning	Model-native reasoning; optional bodyless leader.	Baseline has no explicit decentralized plan agreement; leader serializes a phase and changes information topology.
Evaluation	Durable attempt ledger, provenance, termination reasons, usage and latency accounting.	Twenty worlds still yield wide uncertainty; opportunity-conditioned and partner-generalization metrics are incomplete.
Serving	Native Ollama client, retries, separated reasoning, one endpoint per agent possible.	Readiness checks do not guarantee model load; auto context and parallel slots can cause VRAM failures; provider retries can mask policy latency.

The implementation already contains important safeguards: per-agent clients, parallel worker futures, immutable configuration snapshots, failed-attempt accounting, and communication routing metrics. Improvements should build on those observability hooks instead of adding a second opaque agent loop.

4 Prioritized improvements

4.1 P0: a decentralized commitment protocol

The highest-priority experiment should replace unconstrained coordination prose with an optional typed envelope while retaining a short free-text payload:

```
TYPE: PROPOSE | ACCEPT | REJECT | READY | COMMIT | CANCEL | STATUS
ID: stable task/commitment identifier
TO: agent ids or team
TARGET: entity and coordinate
ACTION: canonical action
WINDOW: earliest/latest tick
PRECONDITIONS: local facts
```

```
PAYLOAD: <= 160 characters
```

A synchronous task uses a two-phase handshake: one agent proposes a target and tick window; partners explicitly accept or reject; all send READY after reaching valid positions; then each commits to the same action/tick. Conflicting commitments are surfaced in the next prompt. Expired commitments are removed automatically. This mechanism remains decentralized: no process chooses the plan, and each message contains only sender-visible information.

Hypothesis. Explicit acknowledgement and stable IDs will improve synchronous-hard opportunity conversion and reduce repeated contradictory messages without increasing total bytes. **Primary metrics.** Proposal acceptance latency, agreement rate, ready-to-execution latency, commitment violations, synchronized-action success per exposed opportunity, delivered bytes, and Coord.%. **Control.** The unchanged free-form broadcast baseline with identical model, prompts outside the protocol block, seeds, and generation parameters.

4.2 P0: structured private plan memory

Replace the optional free-text scratchpad in one ablation with a bounded schema:

```
GOAL: current role-consistent objective
NEXT: up to three state-contingent steps
COMMITMENTS: ids, counterparties, target, tick/window, status
FACTS: off-screen discoveries with source and last-confirmed tick
FAILURES: attempted action, observed outcome, revised belief
```

The harness should reject duplicate observation text, age facts, and cap each section separately. Memory updates should be patches rather than full rewrites, reducing both drift and token cost. This directly tests the paper's inference that planner-like memory is useful while state restatement is not. Measure stale-fact use, commitment recall, plan completion, memory novelty, and tokens in addition to reward.

4.3 P0: opportunity-conditioned diagnostics

Reward coverage alone cannot distinguish "the team never found the site" from "the team found it and failed to coordinate." Instrument a coordination opportunity state machine:

1. opportunity exists in world;
2. at least one agent observes it;
3. information reaches required partners;
4. a compatible plan is proposed;
5. agents arrive in valid positions;
6. agreement is reached;
7. joint action is attempted;
8. environment confirms success.

Report transition probabilities and time in each state by task class. These metrics localize whether an intervention improves discovery, routing, navigation, agreement, or execution. They also align with process-oriented evaluation in Collab-Overcooked and milestone metrics in MultiAgentBench [8, 12].

4.4 P1: deterministic execution aids, not privileged planning

The language model should select goals and commitments, while deterministic tools handle brittle mechanics already derivable from the local observation:

- a local coordinate parser and short-path planner over visible tiles;

- semantic action validation against current affordances;
- a one-step “face target” helper;
- repeated-no-progress detection with obstacle evidence;
- an atomic action formatter that always emits one canonical action.

This is a tool-use treatment, not the canonical zero-shot policy. It should be reported separately because it changes the action scaffold. The tool must receive no hidden state. Compare invalid actions, loops, navigation efficiency, coordination-site arrival, tokens, and final outcomes. ReAct’s grounded reasoning/action cycle motivates this separation [11].

4.5 P1: event-triggered and addressed communication

Broadcasting every tick is expensive and can bury important changes. Add a configurable policy that sends only when an event occurs: new discovery, commitment state change, material inventory or health threshold, failed action, request, or changed goal. Allow direct addressing and local multicast; retain a periodic heartbeat to detect silence. Evaluate fixed byte budgets and induced delay/loss.

This treatment should answer whether communication quality, rather than volume, drives the paper’s ablation result. Report value per delivered byte, redundant-message rate, missed critical events, and team performance. At larger populations, sparse routing is mandatory because all-to-all broadcast grows quadratically.

4.6 P1: failure detection and decentralized reassignment

Introduce seeded crash-stop schedules distinct from in-world death. A crashed agent receives no model call, emits Noop, and communicates nothing. Healthy agents must infer failure from missing heartbeats or unmet commitments, cancel affected agreements, and negotiate takeover. Measure detection delay, abandoned obligations, reassignment latency, duplicated work, and recovered reward.

This provides a sharper test of adaptive collaboration than simply adding more agents. It also prepares the architecture for network partitions and timeouts without prematurely introducing Byzantine behavior.

4.7 P1: partner-generalization matrix

Evaluate each policy with partners sampled across model, prompt, reasoning budget, message style, and competence. Use homogeneous controls, leave-one-partner-family-out tests, and intentionally weak or silent partners. Standardize the protocol schema across models while allowing free-text payloads. Report worst-partner, average-partner, and adaptation performance, not only homogeneous mean. This tests the coordination robustness emphasized by Overcooked and Melting Pot [3, 5] and avoids assuming that one strong model lifts a mixed team.

4.8 P2: skill learning and policy improvement

Once diagnostics identify recurring failures, construct a versioned skill library from successful traces: approach a shared target, stage a handover, execute a synchronous gather, transfer a resource, recover from blocked navigation, and revive a teammate. Skills should specify preconditions, observable termination, and failure exits. Retrieve only relevant skills and test held-out worlds. Voyager supports the general value of executable skill accumulation [10], but AlemDICE should require multi-agent preconditions and commitments rather than copying a single-agent library.

Subsequent training can proceed conservatively: supervised fine-tuning on validated decisions and protocol messages; preference optimization against paired successful/failed coordination traces; then offline or online RL using environment reward plus tightly scoped auxiliary rewards for valid commitments. Auxiliary shaping must not become the headline score, and held-out procedural seeds and partner populations must remain untouched.

5 Evaluation design

5.1 Preserve controls and use a staged funnel

Each proposal starts with deterministic unit tests and a one-seed smoke test, then a three-seed mechanism check, and only then the canonical 20 worlds on Easy, Medium, and Hard. All material comparisons use paired world seeds. The published baseline remains frozen; new profiles include a schema/version identifier and cannot write into baseline result directories.

The canonical metrics remain Team achievement percentage, coordination and normal achievement percentages, action parse rate, reward decomposition, survival, usage, and latency. Add the opportunity funnel, commitment metrics, delivered bytes, and provider health. Do not average the three difficulties into one headline. Report environment outcomes separately from failed attempts, while retaining failed-attempt compute and error counts.

5.2 Uncertainty and decision rules

Twenty episodes are enough for a standard protocol but not necessarily a precise ranking. Report episode-level values, paired differences, bootstrap 95% intervals, and robust aggregates such as the interquartile mean. Avoid declaring a win from point estimates whose intervals overlap materially. This follows the warning that few-run RL comparisons can reverse under appropriate uncertainty analysis [1].

Before running an experiment, register a primary outcome and guardrails. For the commitment protocol, a reasonable primary outcome is Hard synchronous-success per exposed opportunity. Guardrails are Base%, token count, delivered bytes, p95 tick latency, parse rate, and provider failure rate. Promotion requires a positive paired effect on the primary metric with no practically important guardrail regression. Exact thresholds should be fixed after measuring baseline variance, not after seeing treatment results.

5.3 Factorial ablations

Avoid replacing the entire harness at once. A minimal informative design is:

Arm	Protocol	Structured memory	Deterministic tools
A	free broadcast	no	no
B	commitment	no	no
C	free broadcast	yes	no
D	commitment	yes	no
E	commitment	yes	yes

Run A–D first to estimate interaction between communication and memory. Run E only if D shows a coordination gain, because tools otherwise confound whether agreement or mechanical execution improved. Cross these arms with reasoning on/off only for models where the API implements a real reasoning control.

6 Serving and systems improvements

Policy experiments are invalid when infrastructure failures selectively remove agents. The observed four-A100 setup illustrates three hazards: an Ollama server can be reachable yet lack the model; automatic

256K context on an 80GB GPU can inflate the KV cache; and `OLLAMA_NUM_PARALLEL` multiplies context memory [7, 6].

The reliable single-episode topology is one server per GPU, one agent per server, explicit `CUDA_VISIBLE_DEVICES`, a shared read-only model store, explicit `num_ctx`, and `OLLAMA_NUM_PARALLEL=1`. A preflight must query `/api/tags`, preload the exact digest, query `/api/ps`, and execute one bounded response on every endpoint. Record model digest, context allocation, GPU mapping, load time, prompt/eval tokens, time-to-first-token, generation latency, queue time, and retry reason.

For multi-episode throughput, benchmark rather than assume. Compare (i) one Ollama replica per GPU, (ii) a batching-oriented vLLM deployment, and (iii) if supported, tensor-parallel serving. vLLM's PagedAttention was designed to reduce KV-cache waste and increase serving throughput [4]; that does not guarantee a win for this exact Gemma build, so use identical prompts and measure tokens/s, p50/p95 tick latency, failures, energy if available, and output equivalence. Separate leader latency from parallel worker-round latency. Retry storms must be bounded and counted; a 500 response should fail the preflight rather than consume an episode.

7 Recommended implementation sequence

Phase	Deliverable	Exit criterion	Priority
0	Freeze baseline, add per-endpoint preflight and opportunity instrumentation.	Canonical trace unchanged; induced server faults classified before step 0.	**Must**
1	Typed commitment envelopes and parser; no prompt/tool changes otherwise.	Unit tests plus paired smoke traces show correct lifecycle and no hidden-state access.	**Must**
2	Structured patch-based private memory.	Lower duplication and stable commitment recall under long episodes.	**Must**
3	A–D factorial on paired seeds.	Positive Hard opportunity-conversion evidence with uncertainty and guardrails.	**Must**
4	Deterministic local execution aids and event-triggered routing.	Fewer mechanical failures and lower bytes without coordination regression.	**Should**
5	Partner matrix and crash-stop schedules.	Performance characterized across unseen partners and failure fractions.	**Should**
6	Skill library and training from validated trajectories.	Held-out seed/partner gains over strongest prompt-only arm.	**Could**
7	Scale ladder at 8, 16, then 32 agents.	Bounded per-agent context, sparse communication, backpressure, and stable tick latency.	**Could**

8 Threats and non-recommendations

First, changing prompts, memory, tools, and topology simultaneously prevents causal attribution. Second, a bodyless leader can improve a pilot while weakening the claim of decentralized coordination. Third, more reasoning tokens may improve one model and cause another to omit the action; reasoning controls are provider-specific. Fourth, process metrics can be gamed if directly rewarded, so environment outcomes remain primary. Fifth, a single seed is a wiring or qualitative study, not comparative evidence. Sixth, training on canonical evaluation worlds or partners would invalidate generalization claims. Finally, higher GPU utilization is not itself an agent improvement; systems changes must preserve request semantics and be reported separately.

The report does *not* recommend silently replacing the standard baseline, treating message length as communication quality, using hidden world state for navigation, assigning fixed roles from the evaluator, or claiming large-scale collaboration from a crowded fixed map. Those choices would make scores easier to improve while weakening the scientific question.

9 Conclusion

Alem already identifies the right bottleneck: capable individual actors do not reliably form and execute shared plans. The most direct next experiment is therefore a decentralized commitment layer paired with plan-oriented memory and opportunity-conditioned diagnostics. This intervention targets the paper's largest ablation effect, Gemma's observed memory weakness, and the synchronous- hard failure chain without granting a planner privileged state. Rigorous paired-seed evaluation, unseen-partner tests, and fault-aware serving are equally important: without them, apparent gains can be prompt variance, partner overfitting, or infrastructure selection effects. The roadmap above turns these ideas into separable, testable stages while preserving the published control.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, pages 29304–29320, 2021.

[2] Saaket Agashe, Yue Fan, Anthony Reyna, and Xin Eric Wang. LLM-coordination: Evaluating and analyzing multi-agent coordination abilities in large language models. In *Findings of NAACL*, pages 8053–8072, 2025.

[3] Micah Carroll, Rohin Shah, Mark K. Ho, Tom Griffiths, Sanjit Seshia, Pieter Abbeel, and Anca Dragan. On the utility of learning about humans for human–ai coordination. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of SOSP*, pages 611–626, 2023.

[5] Joel Z. Leibo, Edgar A. Dueñez-Guzman, Alexander Vezhnevets, John P. Agapiou, Peter Sunehag, Raphael Koster, Jayd Matyas, Charlie Beattie, Igor Mordatch, and Thore Graepel. Scalable evaluation of multi-agent reinforcement learning with melting pot. In *Proceedings of ICML*, volume 139, pages 6187–6199, 2021.

[6] Ollama. Context length. https://docs.ollama.com/context-length, 2026. Accessed 2026-07-21.

[7] Ollama. Ollama faq: concurrency, context, and multi-gpu loading. https://docs.ollama.com/faq, 2026. Accessed 2026-07-21.

[8] Haochen Sun, Shuwen Zhang, Lujie Niu, Lei Ren, Hao Xu, Hao Fu, Fangkun Zhao, Caixia Yuan, and Xiaojie Wang. Collab-overcooked: Benchmarking and evaluating large language models as collaborative agents. In *Proceedings of EMNLP*, pages 4922–4951, 2025.

[9] Kale-ab Abebe Tessera, Andras Szecsenyi, Cameron Barker, Alexander Rutherford, Davide Paglieri, Aidan Scannell, Henry Gouk, Elliot J. Crowley, Tim Rocktäschel, and Amos Storkey. Benchmarking open-ended multi-agent coordination in language agents. *arXiv preprint arXiv:2606.08340*, 2026.

[10] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024.

[11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[12] Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhailong Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. MultiAgentBench: Evaluating the collaboration and competition of LLM agents. In *Proceedings of ACL*, pages 8580–8622, 2025.